

Persistence Scripts and Wazuh Detection

Digital Forensics and SOC Incident Response Lab

Name	Brian Stauffer
Course	C250 – Intermediate Digital Forensics
Lab Tool Focus	Wazuh SIEM / Windows Endpoint Artifacts
Target System	Windows 11 Target
Date	May 24, 2026

End-to-end lab guide formatted in a NetLab-style walkthrough with objectives, required software, step-by-step tasks, screenshot evidence, verification, and cleanup.

1. Introduction

This lab teaches students how benign persistence artifacts can be created on a Windows endpoint and then detected through Wazuh. Persistence is a common attacker objective because it allows access or execution to survive logoff, reboot, or user activity changes. In this lab, the persistence behavior is simulated safely using commands that write timestamps to local marker files instead of deploying malware.

Students will create four common Windows persistence artifacts: a scheduled task, a registry Run key, a startup folder shortcut, and a local administrator service-style account. Students will then verify the artifacts on the Windows target and correlate the activity in Wazuh using Windows process, event, and endpoint telemetry.

2. What This Lab Is About

The lab focuses on digital forensic artifact correlation. One command or event rarely tells the full story. Students must connect local endpoint evidence with SIEM evidence to explain what happened, when it happened, and why the activity matters from a defensive security perspective.

- Create a safe Windows persistence simulation script.
- Execute the script from an elevated PowerShell session.
- Verify the scheduled task, registry Run key, startup shortcut, local user, group membership, script log, and marker file.
- Search Wazuh for process execution and Windows endpoint events related to the persistence activity.
- Document the attack chain and explain how a SOC analyst would interpret the evidence.
- Clean up all lab artifacts after verification.

3. Learning Objectives

1. Explain why scheduled tasks, Run keys, startup folders, and local admin users are persistence mechanisms.
2. Use PowerShell to safely create and verify endpoint artifacts for a controlled lab.
3. Use Wazuh to locate and interpret endpoint detection data.
4. Correlate host-based evidence with SIEM evidence to build a short forensic timeline.
5. Perform cleanup and verify that the simulated persistence artifacts no longer exist.

4. Required Software, Systems, and Versions

Record the exact versions from the lab environment before starting. The table below lists the required components and the version information that should be captured for grading and repeatability.

Component	Purpose in Lab	Required / Compatible Version	How to Verify Version
Windows 11 Target VM	Endpoint where persistence artifacts are created	Windows 11 Pro or Enterprise, 64-bit	Settings > System > About or winver
PowerShell	Runs the safe persistence simulation script	Windows PowerShell 5.1+ or PowerShell 7+	Run: \$PSVersionTable.PSVersion
Wazuh Manager	Collects and indexes endpoint telemetry	Wazuh 4.x or version used by the class lab	Wazuh dashboard > About / Management
Wazuh Dashboard	Searches and visualizes alerts/events	Same major version as Wazuh manager	Dashboard About page
Wazuh Agent on Windows Target	Forwards Windows telemetry to Wazuh	Same major version family as Wazuh manager	Agent status page or wazuh-agent service

Web Browser	Accesses Wazuh dashboard	Microsoft Edge or Google Chrome current version	Browser Help > About
Velociraptor (Optional)	Additional endpoint verification or collection	Class lab version, if deployed	Velociraptor GUI > Server information
Administrative Account	Required to create task/user and modify system artifacts	Local or domain account with admin rights	Confirm PowerShell is elevated

5. Lab Scenario

You are a SOC analyst and digital forensics student investigating suspicious persistence behavior on a Windows 11 endpoint. The activity is generated intentionally as part of a controlled training exercise. Your job is to execute the safe simulation, collect evidence from the host, confirm Wazuh detection, and explain the forensic significance of each artifact.

6. Safety and Scope Rules

- Run this lab only in an authorized classroom, NetLab, or isolated lab environment.
- Do not run persistence scripts on personal, production, or unauthorized systems.
- The script is benign and only writes marker files, but it still creates real Windows persistence artifacts.
- Complete the cleanup section before ending the lab.

7. Lab Topology

Recommended topology:

- Windows 11 Workstation or jump box used to access lab tools.
- Windows 11 Target running the Wazuh agent.
- Wazuh Manager/Dashboard receiving endpoint events from the target.
- Optional Velociraptor server/client for additional endpoint validation.

8. Lab Procedure

Part 1 - Prepare the Windows Target

Log in to the Windows 11 target with an administrative account. Open PowerShell as Administrator. This is required because the lab creates a scheduled task with elevated privileges and adds a new local user to the Administrators group.

```
mkdir C:\Temp -ErrorAction SilentlyContinue
notepad C:\Temp\phase3-windows-persistence.ps1
```

What this does: The first command creates a working folder for the lab. The second command opens Notepad so the script can be saved in C:\Temp. This matters because keeping scripts and logs in a known path makes verification and cleanup easier.

Part 2 - Create the Safe Windows Persistence Simulation Script

Paste the script below into Notepad and save it as C:\Temp\phase3-windows-persistence.ps1. The script creates four benign persistence artifacts: WindowsTelemetryUpdater scheduled task, OneDriveSecurityUpdate Run key, SecurityHealthCheck.Ink startup shortcut, and svc_backup local administrator account.

```

# phase3-windows-persistence.ps1
# Safe Windows persistence simulation for DFIR lab
# Creates benign persistence artifacts for detection in Wazuh/Velociraptor

$LogPath = "C:\Temp\phase3-windows-persistence.log"
$MarkerPath = "C:\ProgramData\phase3_persistence_marker.txt"

function Write-LabLog {
    param([string]$Message)
    $Time = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    "$Time - $Message" | Tee-Object -FilePath $LogPath -Append
}

Write-LabLog "===== Windows persistence simulation started ====="
New-Item -ItemType Directory -Path "C:\Temp" -Force | Out-Null

$PayloadCommand = "powershell.exe -ExecutionPolicy Bypass -WindowStyle Hidden -Command `\"Add-Content -
Path '$MarkerPath' -Value ('Persistence check ran at ' + (Get-Date))`\""

# 1. Scheduled Task Persistence
try {
    $TaskName = "WindowsTelemetryUpdater"
    schtasks /Create /TN $TaskName /SC ONLOGON /RL HIGHEST /TR $PayloadCommand /F | Out-Null
    Write-LabLog "Created scheduled task persistence: $TaskName"
}
catch { Write-LabLog "ERROR creating scheduled task: $($_.Exception.Message)" }

# 2. Registry Run Key Persistence
try {
    $RunKeyPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
    $RunValueName = "OneDriveSecurityUpdate"
    New-ItemProperty -Path $RunKeyPath -Name $RunValueName -Value $PayloadCommand -PropertyType String
    -Force | Out-Null
    Write-LabLog "Created HKCU Run key persistence: $RunValueName"
}
catch { Write-LabLog "ERROR creating Run key: $($_.Exception.Message)" }

# 3. Startup Folder Persistence
try {
    $StartupFolder = "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup"
    $ShortcutPath = Join-Path $StartupFolder "SecurityHealthCheck.lnk"
    $WScriptShell = New-Object -ComObject WScript.Shell
    $Shortcut = $WScriptShell.CreateShortcut($ShortcutPath)
    $Shortcut.TargetPath = "powershell.exe"
    $Shortcut.Arguments = "-ExecutionPolicy Bypass -WindowStyle Hidden -Command `\"Add-Content -Path
'$MarkerPath' -Value ('Startup folder persistence ran at ' + (Get-Date))`\""
    $Shortcut.WorkingDirectory = "C:\Windows\System32"
    $Shortcut.Save()
    Write-LabLog "Created Startup folder shortcut persistence: $ShortcutPath"
}
catch { Write-LabLog "ERROR creating Startup folder shortcut: $($_.Exception.Message)" }

# 4. New Local Admin User Persistence
try {
    $Username = "svc_backup"
    $Password = "P@ssw0rd!Lab2026"

```

```

$SecurePassword = ConvertTo-SecureString $Password -AsPlainText -Force
if (-not (Get-LocalUser -Name $Username -ErrorAction SilentlyContinue)) {
    New-LocalUser -Name $Username -Password $SecurePassword -FullName "Backup Service Account" -
Description "SOC lab persistence simulation account" | Out-Null
    Write-LabLog "Created local user: $Username"
}
else { Write-LabLog "Local user already exists: $Username" }
Add-LocalGroupMember -Group "Administrators" -Member $Username -ErrorAction SilentlyContinue
Write-LabLog "Added user to local Administrators group: $Username"
}
catch { Write-LabLog "ERROR creating local admin user: $($_.Exception.Message)" }

Write-LabLog "==== Windows persistence simulation completed ====="
Write-LabLog "Marker file location: $MarkerPath"
Write-LabLog "Log file location: $LogPath"

```

Save and close notepad.

Part 3 - Run the Script and Record Start/Finish Time

Before running the script, record the start time. After the script completes, record the finish time. These times will be used to search Wazuh and correlate host activity with SIEM activity.

```

Get-Date
Set-ExecutionPolicy Bypass -Scope Process -Force
powershell.exe -ExecutionPolicy Bypass -File C:\Temp\phase3-windows-persistence.ps1
Get-Date

```

Expected result: The script should complete without error and create a log file at C:\Temp\phase3-windows-persistence.log. The log should show the scheduled task, Run key, startup shortcut, local user, and administrator group membership creation steps.

Save and close Notepad.

3. Run the script

From the elevated PowerShell window:

```
PS C:\windows\system32> Set-ExecutionPolicy Bypass -Scope Process -Force powershell.exe -ExecutionPolicy Bypass -File C:\Temp\phase3-windows-persistence.ps1
```

4. Record start and finish time

Right before you run it:

```
PS C:\windows\system32> Get-Date
```

Right after it finishes:

```
PS C:\windows\system32> Get-Date
```

Record both times in your notes so you can correlate events in **Wazuh** and **Velociraptor**.

Example note:

Windows persistence script started: 2026-05-14 15:22

Windows persistence script completed: 2026-05-14 15:24

5. Minimum local checks after completion

Run these commands to confirm each artifact exists.

Check the scheduled task

```
PS C:\windows\system32> schtasks /Query /TN "windowsTelemetryUpdater" /V /FO LIST
```

You should see the task name and the command it runs.

Check the Run key

```
PS C:\windows\system32> Get-ItemProperty "HKCU:\Software\Microsoft\windows\CurrentVersion\Run" | select-object OneDriveSecurityUpdate
```

You should see a value named:

OneDriveSecurityUpdate

Check the Startup folder shortcut

```
PS C:\windows\system32> Get-ChildItem "$env:APPDATA\Microsoft\windows\Start Menu\Programs\Startup"
```

You should see:

SecurityHealthCheck.lnk

Check the new local admin user

```
PS C:\windows\system32> Get-LocalUser svc_backup
```

Then check admin group membership:

```
PS C:\windows\system32> Get-LocalGroupMember Administrators
```

You should see:

svc_backup

Check the script log

```
PS C:\windows\system32> type C:\Temp\phase3-windows-persistence.log
```

Figure 1. Script execution and required local verification commands from the lab instructions. The screenshot shows the commands students use to run the script, record timestamps, and begin checking the created persistence artifacts.

Part 4 - Verify Persistence Artifacts on the Windows Target

Use the following checks to confirm each artifact exists. Each command validates one part of the persistence simulation and creates host evidence that can later be compared against Wazuh telemetry.

Check the scheduled task

```
schtasks /Query /TN "WindowsTelemetryUpdater" /V /FO LIST
```

Why it matters: This confirms that the WindowsTelemetryUpdater scheduled task exists and shows the command that will run when the task is triggered.

Check the Run key

```
Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" | Select-Object  
OneDriveSecurityUpdate
```

Why it matters: This confirms the OneDriveSecurityUpdate registry value exists in the current user Run key.

Check the Startup folder shortcut

```
Get-ChildItem "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup"
```

Why it matters: This confirms SecurityHealthCheck.lnk exists in the user startup folder.

Check the local user

```
Get-LocalUser svc_backup
```

Why it matters: This confirms that the svc_backup local account was created.

Check local Administrators membership

```
Get-LocalGroupMember Administrators
```

Why it matters: This confirms that svc_backup was added to the local Administrators group.

Check the script log

```
type C:\Temp\phase3-windows-persistence.log
```

Why it matters: This confirms the script wrote an execution log that can be used as local evidence.

Trigger and check the marker file

```
schtasks /Run /TN "WindowsTelemetryUpdater"  
type C:\ProgramData\phase3_persistence_marker.txt
```

Why it matters: This manually runs the scheduled task and confirms the benign payload wrote a timestamp marker file.

```

PS C:\Users\Administrator> type C:\Temp\phase3-windows-persistence.log
2026-05-14 15:31:50 - ===== Windows persistence simulation started =====
2026-05-14 15:31:50 - Created scheduled task persistence: WindowsTelemetryUpdater
2026-05-14 15:31:50 - Created HKCU Run key persistence: OneDriveSecurityUpdate
2026-05-14 15:31:51 - Created Startup folder shortcut persistence: C:\Users\Administrator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\SecurityHealthCheck.lnk
2026-05-14 15:31:51 - Created local user: svc_backup
2026-05-14 15:31:51 - Added user to local Administrators group: svc_backup
2026-05-14 15:31:51 - ===== Windows persistence simulation completed =====
2026-05-14 15:31:51 - Marker file location: C:\ProgramData\phase3_persistence_marker.txt
2026-05-14 15:31:51 - Log file location: C:\Temp\phase3-windows-persistence.log
PS C:\Users\Administrator>

```

Check the marker file

The marker file may not populate until the task or startup item runs. You can manually trigger the scheduled task:

```
PS C:\Windows\system32> schtasks /Run /TN "WindowsTelemetryUpdater"
```

Then check:

```
PS C:\Windows\system32> type C:\ProgramData\phase3_persistence_marker.txt
```

WIN11 Target host verification

```

PS C:\Users\Administrator> Start-ScheduledTask -TaskName "WindowsTelemetryUpdater"
PS C:\Users\Administrator> type C:\ProgramData\phase3_persistence_marker.txt
Persistence check ran at 05/14/2026 15:46:47
Persistence check ran at 05/14/2026 15:51:58
PS C:\Users\Administrator> Get-ScheduledTask -TaskName "WindowsTelemetryUpdater"

TaskPath          TaskName          State
-----
\                  WindowsTelemetryUpdater Ready

PS C:\Users\Administrator> Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" | Select-Object OneDriveSecurityUpdate
OneDriveSecurityUpdate
powershell.exe -ExecutionPolicy Bypass -WindowStyle Hidden -Command "Add-Content -Path 'C:\ProgramData\phase3_persis..."

PS C:\Users\Administrator> Get-ChildItem "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup"

Directory: C:\Users\Administrator\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

Mode                LastWriteTime         Length Name
----                -
-a-----          5/14/2026   3:49 PM           1362 SecurityHealthCheck.lnk

PS C:\Users\Administrator> Get-LocalGroupMember Administrators | findstr /i "svc_backup"
User      DESKTOP-EGW4375\svc_backup Local
PS C:\Users\Administrator> Get-ScheduledTask -TaskName "WindowsTelemetryUpdater"

TaskPath          TaskName          State
-----
\                  WindowsTelemetryUpdater Ready

PS C:\Users\Administrator>

```

Figure 2. Windows 11 target host verification. The screenshot shows the local evidence that the scheduled task, Run key, startup shortcut, local administrator account, and marker file were created successfully.

Part 5 - Verify Detection in Wazuh

Open the Wazuh dashboard and search the Windows target events around the recorded start and finish time. Use the target agent name, host name, process name, and artifact names to narrow the results.

Purpose	Example Wazuh Query	Expected Evidence
Search for PowerShell script execution	agent.name:"WIN11-Target" AND data.win.eventdata.processName:*powershell*	Events near the recorded script execution time
Search for scheduled task creation	agent.name:"WIN11-Target" AND (data.win.eventdata.commandLine:*schtasks* OR data.win.eventdata.commandLine:*WindowsTelemetryUpdater*)	Events near the recorded script execution time
Search for the Run key artifact	agent.name:"WIN11-Target" AND data.win.eventdata.commandLine:*OneDriveSecurityUpdate*	Events near the recorded script execution time
Search for marker file activity	agent.name:"WIN11-Target" AND data.win.eventdata.commandLine:*phase3_persistence_marker.txt*	Events near the recorded script execution time
Search for the local account artifact	agent.name:"WIN11-Target" AND (svc_backup OR "Backup Service Account")	Events near the recorded script execution time

Expected Wazuh evidence includes PowerShell execution of the script, schtasks.exe activity for WindowsTelemetryUpdater, scheduled task XML or registration content, PowerShell payload execution, and marker file writes to C:\ProgramData.

Wazuh verification



15:49:43 — schtasks.exe /Create /TN WindowsTelemetryUpdater /SC ONLOGON /RL HIGHEST /TR ...
15:49:43 — powershell.exe -ExecutionPolicy Bypass -File C:\Temp\phase3-windows-persistence.ps1
15:47:55 — schtasks.exe /Query /TN WindowsTelemetryUpdater /V /FO LIST
15:46:47 — powershell.exe ... Add-Content -Path C:\ProgramData\phase3_persistence_marker.txt
15:46:27 — scheduled task XML content for WindowsTelemetryUpdater

PowerShell script executed

↓

schtasks.exe attempted/created scheduled task

↓

WindowsTelemetryUpdater task registered

↓

PowerShell payload executed

↓

Marker file written to C:\ProgramData

Windows cleanup

Remove the scheduled task:

```
PS C:\windows\system32> schtasks /Delete /TN "LabPersistenceTask" /F
```

Remove the proof file if it exists:

```
PS C:\windows\system32> del C:\Temp\lab_persistence.txt
```

Verify:

```
PS C:\windows\system32> schtasks /Query /TN "LabPersistenceTask"
```

You should see an error saying the task does not exist.

Figure 3. Wazuh verification. The screenshot shows endpoint event evidence tied to the script execution, scheduled task activity, PowerShell payload execution, and marker file write activity.

Part 6 - Build a Short Forensic Timeline

Using the local script log, PowerShell timestamps, and Wazuh event timestamps, fill in the timeline below. The goal is to show that the host artifacts and SIEM evidence describe the same activity.

Approx. Time	Evidence Source	Observed Activity	Forensic Meaning
15:49:43	Wazuh	powershell.exe ran phase3-windows-persistence.ps1	Script execution started on the Windows target.
15:49:43	Wazuh	schtasks.exe created WindowsTelemetryUpdater	Scheduled task persistence was attempted or created.
15:46:27	Wazuh	Scheduled task XML content observed	Task registration details were captured.
15:46:47	Wazuh	PowerShell Add-Content wrote to phase3_persistence_marker.txt	Benign payload executed and wrote marker evidence.
Recorded by student	Host log	phase3-windows-persistence.log shows completed actions	Local host log supports the SIEM timeline.

Part 7 - Student Analysis Questions

- Which persistence artifact would likely execute at user login?
- Which artifact modifies the current user registry hive?
- Why is creating a new local administrator account a high-risk persistence behavior?
- Which Wazuh evidence best proves the script executed?
- Which Wazuh evidence best proves the scheduled task was created?
- How would the timeline change if the scheduled task was triggered after the original script finished?
- What cleanup steps are required to return the target to a clean state?

Part 8 - Cleanup and Final Verification

Run the cleanup commands from an elevated PowerShell session. These commands remove the artifacts created by the lab.

```
schtasks /Delete /TN "WindowsTelemetryUpdater" /F
Remove-ItemProperty -Path "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" -Name
"OneDriveSecurityUpdate" -ErrorAction SilentlyContinue
Remove-Item "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup\SecurityHealthCheck.lnk" -Force
-ErrorAction SilentlyContinue
Remove-LocalGroupMember -Group "Administrators" -Member "svc_backup" -ErrorAction SilentlyContinue
Remove-LocalUser -Name "svc_backup" -ErrorAction SilentlyContinue
Remove-Item "C:\ProgramData\phase3_persistence_marker.txt" -Force -ErrorAction SilentlyContinue
Remove-Item "C:\Temp\phase3-windows-persistence.ps1" -Force -ErrorAction SilentlyContinue
Remove-Item "C:\Temp\phase3-windows-persistence.log" -Force -ErrorAction SilentlyContinue
```

Verify cleanup:

```
schtasks /Query /TN "WindowsTelemetryUpdater"
Get-ItemProperty "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run" | Select-Object
OneDriveSecurityUpdate
Get-ChildItem "$env:APPDATA\Microsoft\Windows\Start Menu\Programs\Startup"
Get-LocalUser svc_backup
Get-LocalGroupMember Administrators | findstr /i "svc_backup"
```

Expected result: The scheduled task query should return an error stating the task does not exist. The Run key value, startup shortcut, marker file, and svc_backup account should no longer be present.

9. Required Student Deliverables

- Screenshot showing the script saved or executed from C:\Temp.
- Screenshot showing the script log after execution.
- Screenshot showing scheduled task verification.
- Screenshot showing Run key verification.
- Screenshot showing Startup folder shortcut verification.
- Screenshot showing svc_backup user and Administrators membership verification.
- Screenshot showing Wazuh events for PowerShell and/or schtasks.exe activity.
- Completed forensic timeline table.
- Answers to the student analysis questions.
- Screenshot or notes proving cleanup was completed.

10. Lab Summary

In this lab, students safely simulated Windows persistence and used Wazuh to detect and explain the activity. The scheduled task, Run key, startup folder shortcut, and local administrator account each represent a different way attackers may attempt to maintain access. By comparing local host checks with Wazuh telemetry, students practiced artifact correlation, timeline building, and cleanup verification in a realistic DFIR workflow.

End of Lab